Bare-Metal UART, I²C and SPI Drivers for an STM32 Cortex-M: Register-Level Implementation and Electrical Verification

Clara Ureche
BEng (Hons) Electrical and Electronic Engineering, Lancaster University

Report type	Technical report
Status	Implemented and hardware-tested against external ICs
Platform	STM32 Cortex-M; register listings shown for STM32F4 at 16 MHz HSI
Tools	C (no HAL), CMSIS device header, oscilloscope, logic analyser
Date	2026

Abstract. Drivers for UART, I²C and SPI were written in C for an STM32 Cortex-M microcontroller without the vendor hardware abstraction layer. All configuration and data transfer use direct access to peripheral control, status and data registers, derived from the reference manual. The objective was to implement the three interfaces at register level and confirm at the electrical interface that bus behaviour matches the specification. Each driver follows the same sequence: enable the peripheral clock, configure GPIO alternate functions, program the peripheral, then transfer under status-flag control. Verification used an oscilloscope for levels and clock timing and a logic analyser to decode complete transactions against real slave devices on shared buses. Configured parameters (115200 baud 8N1, 100 kHz I²C, 1 MHz SPI mode 0) are supported by calculation; for example, the programmed UART divider gives a baud error of 0.64%.

Index terms: bare-metal firmware, ARM Cortex-M, register-level programming, UART, I²C, SPI, logic analyser.

1. Objective

Vendor HALs hide clock configuration, pin multiplexing and peripheral state machines, which limits visibility when a transaction fails. This project implements each driver from the reference manual [1] so that every register write is explicit, and validates the result on the physical bus rather than in software loopback.

Table 1. Interfaces implemented.

Interface	Clocking	Signals	Addressing	Error indication	Verification
UART	Asynchronous	TX, RX	None	FE, PE, ORE flags	Oscilloscope, logic analyser
I²C	Synchronous, open drain	SCL, SDA	7-bit address	ACK/NACK	Logic analyser
SPI	Synchronous, full duplex	SCK, MOSI, MISO, CS	Chip select	None at bus level	Oscilloscope, logic analyser

2. Peripheral Bring-Up

Peripheral clocks are gated off at reset, so each driver first enables its clock in RCC (AHB1ENR for GPIO, APB1ENR or APB2ENR for the serial peripherals). Writes to a clock-gated peripheral have no effect. Pins are then switched to alternate-function mode in MODER, the function is selected in AFR, and OSPEEDR,

OTYPER and PUPDR are set as required. I^{2}C requires open-drain outputs with pull-ups; push-pull outputs may appear to work at low speed but violate the bus specification [3].

3. UART

$$USARTDIV = f_{CK} / (16 \cdot baud) \tag{1}$$

The divider is written to BRR as a mantissa and 4-bit fraction. The factor 16 is the receiver oversampling ratio. Word length, parity and stop bits are set in CR1 and CR2, and UE, TE and RE enable the peripheral. The driver polls TXE before writing DR and RXNE before reading, and checks ORE, FE and PE. With f_{CK} = 16 MHz, BRR = 138 (USARTDIV = 8.625) gives 115 942 baud, an error of +0.64%, within the approximately 2% tolerance of 16x oversampling.

4. I^{2}C

The 7-bit master transaction follows the reference-manual sequence, with every step gated on a status flag so that it tolerates bus loading and clock stretching:

1. Set START; wait for SB.
2. Write the address and R/W bit to DR; wait for ADDR; clear it by reading SR1 then SR2.
3. Transfer data, polling TXE (write) or RXNE (read), with NACK on the final received byte.
4. Set STOP.

With CR2 = 16 (MHz), CCR = 80 gives $t_{high} = t_{low}$ = 80 x 62.5 ns = 5 µs, i.e. 100 kHz standard mode, and TRISE = 17 corresponds to the 1000 ns maximum rise time plus one.

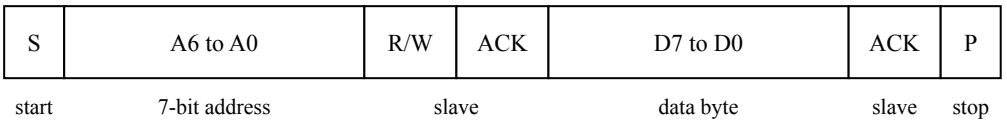


Figure 1. Field structure of a single-byte I^{2}C write as decoded on the logic analyser and compared with the firmware sequence.

5. SPI

SPI1 is configured in CR1 as full-duplex master, MSB first, with prescaler BR = 3 (f_{PCLK}/16 = 1 MHz) and CPOL and CPHA matched to the slave datasheet. Chip select is driven as a GPIO so multi-byte transactions can be framed, and BSY is checked before de-asserting it. An incorrect mode shifts the sampling edge by half a clock period, so the mode was confirmed on the oscilloscope.

Table 2. SPI modes. Mode 0 was used.

Mode	CPOL	CPHA	Idle clock	Data sampled on
0	0	0	Low	Rising (leading) edge
1	0	1	Low	Falling (trailing) edge
2	1	0	High	Falling (leading) edge
3	1	1	High	Rising (trailing) edge

6. Configuration

Table 3. Pin assignment (STM32F4 Nucleo and Discovery mapping).

Peripheral	Pins	Configuration
USART2	PA2 TX, PA3 RX	AF7; 115200 baud, 8N1; TXE/RXNE polling
I2C1	PB6 SCL, PB7 SDA	AF4; open drain with pull-ups; 100 kHz
SPI1	PA5 SCK, PA6 MISO, PA7 MOSI	AF5; mode 0; $f_{PCLK}/16$; software NSS

7. Verification

Two instruments were used. The oscilloscope checked analogue properties: signal levels, rise and fall times, clock frequency and idle states. The logic analyser captured and decoded complete transactions: UART framing and baud rate, I^2C address and acknowledge sequence (Figure 1), and SPI clock-phase alignment. Observing the bus directly distinguishes firmware faults (incorrect register values) from hardware faults (missing pull-ups, wiring errors, wrong alternate function). Each driver was tested against external slave ICs on shared buses, which confirms timing, levels, addressing and acknowledgement against devices not designed by the author.

Table 4. Calculated per-byte transfer time at the configured clocks.

Bus	Clock	Time per byte	CPU load (polled)
USART2	115200 baud, 8N1	Approx. 87 µs	100% during transfer
I2C1	100 kHz	Approx. 90 µs	100% during transfer
SPI1	1 MHz	Approx. 8 µs	100% during transfer

8. Limitations

- Captured waveforms and decoded traces are not reproduced in this report, and the slave ICs are not identified.
- Maximum reliable baud rate, bus frequency and error rates were not measured.
- The drivers are polled and blocking; they occupy the CPU for the full transfer.
- Bus-fault recovery (for example, a slave holding SDA low) is not implemented.

9. Conclusion and Future Work

Register-level drivers for the three interfaces were implemented and verified on the physical bus. The bring-up sequence is common to all three and transfers across STM32 families with changes only to bit positions. Future work: interrupt- and DMA-driven variants (including UART IDLE-line detection for variable-length frames); I^2C bus recovery by clocking SCL up to nine times followed by STOP; measurement of maximum reliable rates; and inclusion of annotated captures for each protocol.

Appendix A. Register-Level Code (STM32F4)

```
#include "stm32f4xx.h"

void uart2_init(void) {                    /* PA2 TX, PA3 RX, 115200 at 16 MHz */
   RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
   RCC->APB1ENR |= RCC_APB1ENR_USART2EN;
   GPIOA->MODER  |= (2u << (2*2)) | (2u << (2*3));     /* alternate function */
   GPIOA->AFR[0] |= (7u << (4*2)) | (7u << (4*3));     /* AF7 = USART2 */
   USART2->BRR = 16000000u / 115200u;
   USART2->CR1 = USART_CR1_TE | USART_CR1_RE | USART_CR1_UE;
}
void uart2_putc(uint8_t c) { while (!(USART2->SR & USART_SR_TXE)); USART2->DR = c; }

void i2c1_write(uint8_t addr, uint8_t reg, uint8_t val) {
   while (I2C1->SR2 & I2C_SR2_BUSY);
   I2C1->CR1 |= I2C_CR1_START;   while (!(I2C1->SR1 & I2C_SR1_SB));
   I2C1->DR = (addr << 1);       while (!(I2C1->SR1 & I2C_SR1_ADDR));
   (void)I2C1->SR2;                           /* clear ADDR */
   I2C1->DR = reg;               while (!(I2C1->SR1 & I2C_SR1_TXE));
   I2C1->DR = val;               while (!(I2C1->SR1 & I2C_SR1_BTF));
   I2C1->CR1 |= I2C_CR1_STOP;
}

uint8_t spi1_xfer(uint8_t d) {
   while (!(SPI1->SR & SPI_SR_TXE));  SPI1->DR = d;
   while (!(SPI1->SR & SPI_SR_RXNE)); return (uint8_t)SPI1->DR;
}
```

Listing A1. Representative excerpt (I2C and SPI initialisation omitted; see Sections 4 and 5).

References

[1] STMicroelectronics, "RM0090: STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 reference manual."

[2] J. Yiu, *The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors*, 3rd ed. Newnes, 2014.

[3] NXP Semiconductors, "I^{2}C-bus specification and user manual," UM10204, rev. 7, 2021.

[4] ARM, "Cortex-M4 Devices Generic User Guide," DUI 0553.